241801053-J B Dhejasvi Unconstrained Optimiztion problems

Task: Write a Python code to find extreme value for each non-linear functions. Also plot the graph and locate extreme values

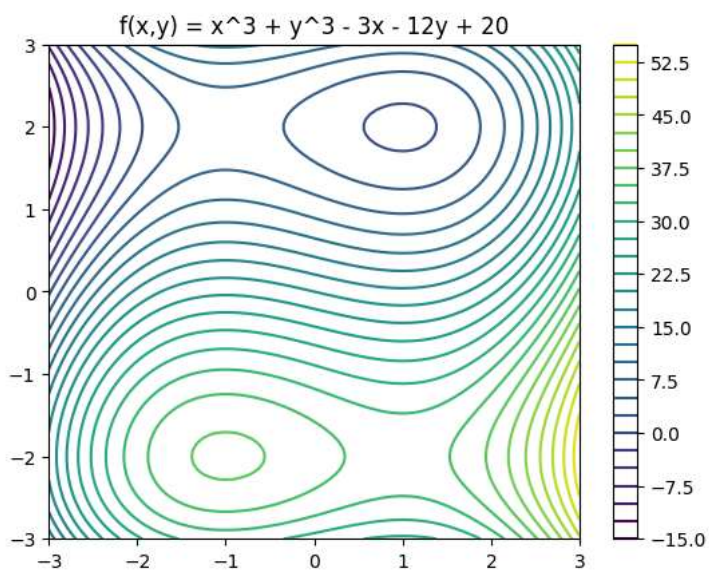
1. $f(x) = x\ 3 + y\ 3 - 3x - 12y + 20$
2. $f(x) = 2x\ 3 - 3x\ 2 - 12x + 10$
3. $f(x) = x\ 3 - 3x + y\ 3 - 3y$
4. $f(x) = x\ 4 + y\ 4 - 4xy + 1$

```
import numpy as np
import matplotlib.pyplot as plt

def f(x, y):
    return x**3 + y**3 - 3*x - 12*y + 20

x = np.linspace(-3, 3, 100)
y = np.linspace(-3, 3, 100)
X, Y = np.meshgrid(x, y)
Z = f(X, Y)

plt.contour(X, Y, Z, levels=30)
plt.title("f(x,y) = x^3 + y^3 - 3x - 12y + 20")
plt.colorbar()
plt.show()
```

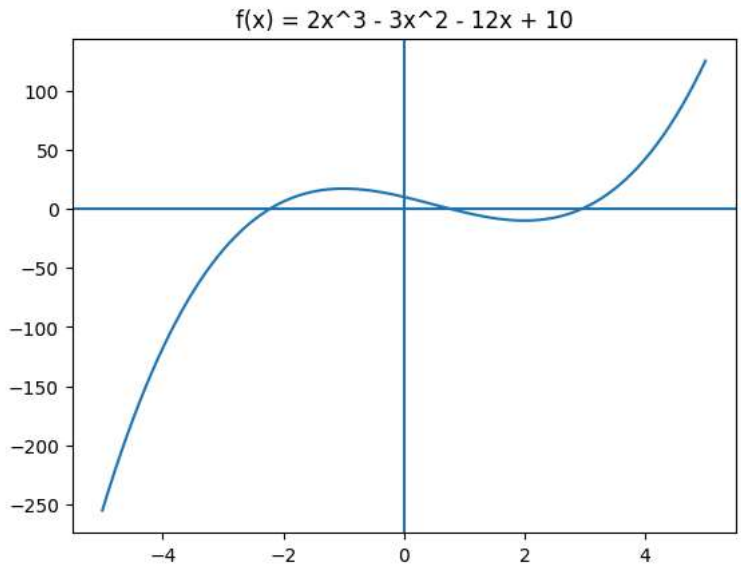



```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return 2*x**3 - 3*x**2 - 12*x + 10

x = np.linspace(-5, 5, 100)
y = f(x)

plt.plot(x, y)
plt.title("f(x) = 2x^3 - 3x^2 - 12x + 10")
plt.axhline(0)
plt.axvline(0)
plt.show()
```

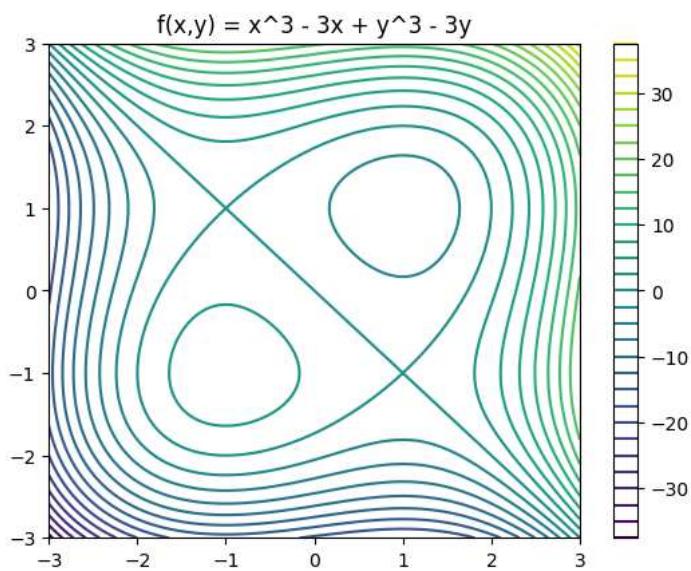



```
import numpy as np
import matplotlib.pyplot as plt

def f(x, y):
    return x**3 - 3*x + y**3 - 3*y

x = np.linspace(-3, 3, 100)
y = np.linspace(-3, 3, 100)
X, Y = np.meshgrid(x, y)
Z = f(X, Y)

plt.contour(X, Y, Z, levels=30)
plt.title("f(x,y) = x^3 - 3x + y^3 - 3y")
plt.colorbar()
plt.show()
```

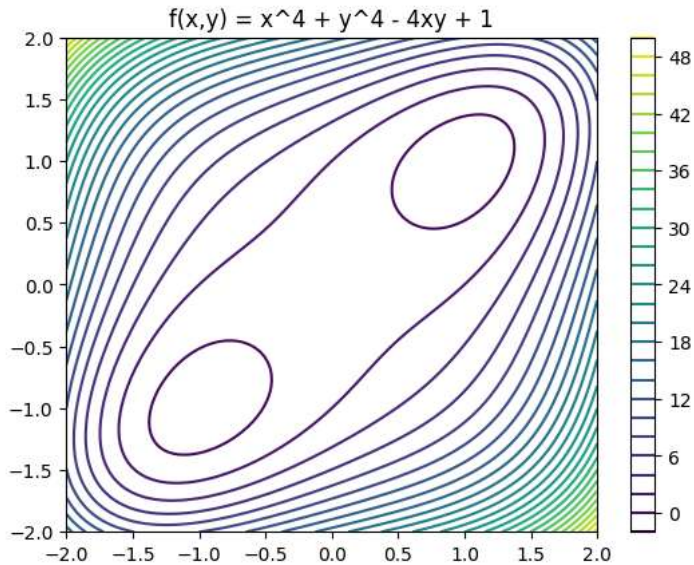



```
import numpy as np
import matplotlib.pyplot as plt

def f(x, y):
    return x**4 + y**4 - 4*x*y + 1

x = np.linspace(-2, 2, 100)
y = np.linspace(-2, 2, 100)
X, Y = np.meshgrid(x, y)
Z = f(X, Y)
```

```
plt.contour(X, Y, Z, levels=30)
plt.title("f(x,y) = x^4 + y^4 - 4xy + 1")
plt.colorbar()
plt.show()
```



Using Newton-Raphson method find extreme values

1. min $f(x)$ = x 4 – 4x 2 + 3
2. min $f(x)$ = x 2 + e –x
3. min $f(x)$ = x 2 + 16 x ; x > 0
4. max $f(x)$ = 10 + 8x 2 – x 4
5. Find local maximum and minimum for $f(x)$ = x 3 – 6x 2 + 9x + 5

```
def newton_method(df, ddf, x0, tol=1e-6, max_iter=100):
      x = x0
      for _ in range(max_iter):
          x_new = x - df(x)/ddf(x)
          if abs(x_new - x) < tol:
                 return x_new
          x = x_new
      return x
```

```
def df(x): return 4*x**3 - 8*x
def ddf(x): return 12*x**2 - 8

root = newton_method(df, ddf, 1)
print(root)
```

```
1.4142135623735754
```

```
import numpy as np

def df(x): return 2*x - np.exp(-x)
def ddf(x): return 2 + np.exp(-x)

print(newton_method(df, ddf, 0.5))
```

```
0.3517337112490805
```

```
def df(x): return 2*x - 16/x**2
def ddf(x): return 2 + 32/x**3

print(newton_method(df, ddf, 2))
```

```
2.0
```

```
def df(x): return 16*x - 4*x**3
def ddf(x): return 16 - 12*x**2
```

```
print(newton_method(df, ddf, 2))
```

```
2.0
```

```
def df(x): return 3*x**2 - 12*x + 9
def ddf(x): return 6*x - 12

print("x =", newton_method(df, ddf, 0))  # near 1
print("x =", newton_method(df, ddf, 4))  # near 3
```

```
x = 0.9999999999999989
x = 3.0000000000000004
```